COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 6, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 QUEUES

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA (main.cpp)

main.cpp ILOA | main.cpp ILOB | queueLL.h

ILOA > main.cpp > display(std::queue<char>)

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    // create an object
    std::queue<char> myQ;

    myQ.push('F');
    myQ.push('E');
    myQ.push('N');
    myQ.push('Z');
    myQ.push('I');
    myQ.push('E');
    std::cout << "======================" << std::endl;
    // display
    display(myQ);
    std::cout << "======================" << std::endl;
    // try all member functions:

    // empty
    if (myQ.empty()){
        std::cout<<"The Queue is Empty." << std::endl;
    }else{
        std::cout<< "The Queue is not Empty" << std::endl;
    }

    // size
    std::cout<<"The Queue Size is: " << myQ.size() << std::endl;
```

main.cpp ILOA | main.cpp ILOB | queueLL.h

ILOA > main.cpp > display(std::queue<char>)

```cpp
int main(){

    // front
    std::cout << "The Front Queue is: " << myQ.front() << std::endl;

    // back
    std::cout << "The Back Queue is: " << myQ.back() << std::endl;

    std::cout << "======================" << std::endl;
    // pop
    myQ.pop();
    std::cout<<"Popped Queue: " << std::endl;
    display(myQ);

    // push
    myQ.push('F');
    std::cout<<"Pushed Queue: " << std::endl;
    display(myQ);
    std::cout << "======================" << std::endl;

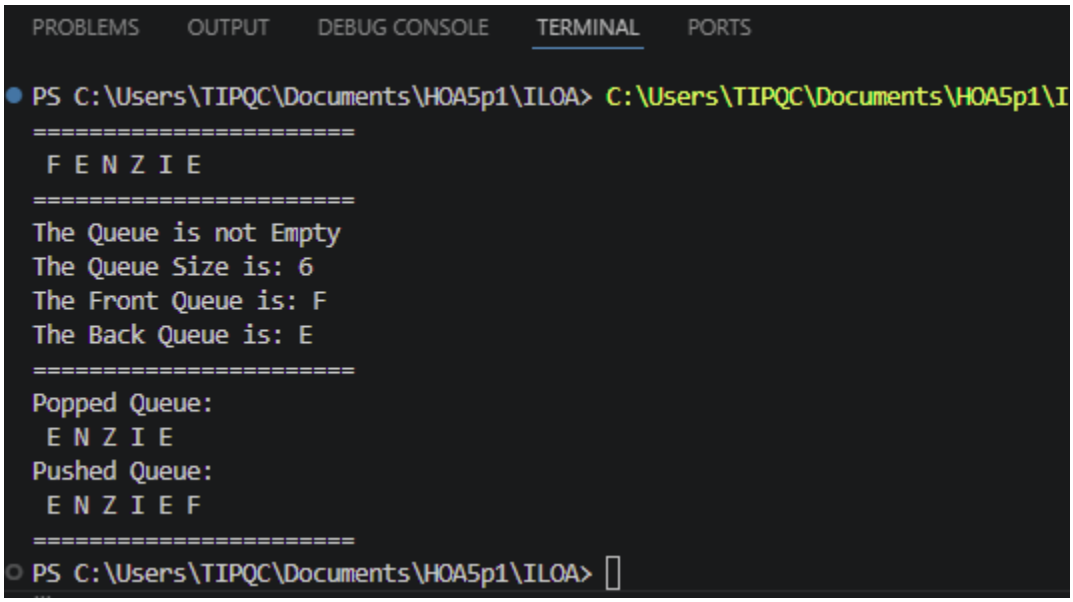

    return 0;
}
```

```cpp

void display(std::queue<char> copyQ){
    // creates a copy of the queue that was passed
    std::queue<char> c = copyQ;
    while(!c.empty()){
        std::cout<<" " << c.front();
        c.pop();
    }
    std::cout<<"\n";

}
```

OUTPUT:



Observation:

In this code, it highlights the use of popping and pushing, while displaying each of these instances manually. In order for the queue to work in this code, we include at the header part the queue and implement it by using the function “myQ. …();“.To check whether the queue is empty, I just simply use the if else statement, and to display it all, i just use the void function while implementing the use of while loop to the system.

ILOB (queueLL.h)

main.cpp ILOA | main.cpp ILOB | queueLL.h

ILOB > queueLL.h > Front<T>(qNode<T>**, qNode<T>**)

```
1   #ifndef QUEUELL_H
2   #define QUEUELL_H
3
4
5   // create a class the node
6   template <typename T>
7   class qNode {
8       public:
9           T data;
10          qNode<T> *next;
11  };
12
13  // creates the new node
14  template <typename T>
15  qNode<T>* new_node(T newData){
16      qNode<T>* newNode = new qNode<T>;
17      newNode->data = newData;
18      newNode->next = nullptr;
19      return newNode;
20  }
21
22  //enqueue operation
23  template <typename T>
24  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
25      // create the new node
26      qNode<T>* newNode = new_node(newData);
27
28      // inserting into an empty queue
29      if((*frontPtr)==nullptr && (*backPtr)==nullptr){
30          (*frontPtr) = newNode;
31          (*backPtr) = newNode;
32      }
```

main.cpp ILOA | main.cpp ILOB | queueLL.h

ILOB > queueLL.h > Front<T>(qNode<T>**, qNode<T>**)

```
1   #ifndef QUEUELL_H
24  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
29      if((*frontPtr)==nullptr && (*backPtr)==nullptr){
33      // inserting an item into a non empty queue
34      (*backPtr)->next = newNode;
35      (*backPtr) = newNode;
36  }
37
38  template <typename T>
39  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
40      // add a check if the queue is already empty
41      if((*frontPtr) == nullptr && (*backPtr)==nullptr){
42          std::cout << "The Queue is Empty!" << std::endl;
43          return;
44      }
45
46      //create a temporary node
47      qNode<T>* deleteNode = nullptr; // temp variable
48      deleteNode = *frontPtr;
49
50      deleteNode = (*frontPtr);
51      // check if the queue has only 1 node
52      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
53          (*frontPtr) = nullptr;
54          (*backPtr) = nullptr;
55          delete deleteNode;
56          return;
57      }
```

main.cpp ILOA | main.cpp ILOB | queueLL.h

ILOB > queueLL.h > Front<T>(qNode<T>**, qNode<T>**)

```
1   #ifndef QUEUELL_H
39  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
58
59      // delete the node
60      (*frontPtr) = (*frontPtr)->next;
61      deleteNode->next = nullptr;
62      delete deleteNode;
63
64  }
65
66  // checking the front
67  template <typename T>
68  void Front(qNode<T>** frontPtr, qNode<T>** backPtr){
69      if((*frontPtr) == nullptr && (*backPtr)==nullptr){
70          std::cout << "The Queue is Empty!" << std::endl;
71          return;
72      }
73      std::cout<< "Checking the Front: " << (*frontPtr)->data << std::endl;
74      }
75
76  // check the back
77  template <typename T>
78  void Back(qNode<T>** frontPtr, qNode<T>** backPtr){
79      if((*frontPtr) == nullptr && (*backPtr)==nullptr){
80          std::cout << "The Queue is Empty!" << std::endl;
81          return;
82      }
83      std::cout<< "Checking the Front: " << (*backPtr)->data << std::endl;
84      }
```

```
85
86
87  // display all
88  template <typename T>
89  void display(qNode<T>** frontPtr, qNode<T>** backPtr){
90      if((*frontPtr) == nullptr && (*backPtr)==nullptr){
91          std::cout << "The Queue is Empty!" << std::endl;
92          return;
93      }
94
95      qNode<T>* current = *frontPtr;
96      std::cout << "Displaying Queue Elements: ";
97      while (current != nullptr) {
98          std::cout << current->data << " ";
99          current = current->next;
100     }
101     std::cout << std::endl;
102 }
103
104 #endif
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\Documents\HOA5p1\ILOB> C:\Users\TIPQC\Documents\HOA5p1\ILOB\main.exe
Checking the Back: N
Displaying Queue Elements: F E N
PS C:\Users\TIPQC\Documents\HOA5p1\ILOB> C:\Users\TIPQC\Documents\HOA5p1\ILOB\main.exe
F F
F E
F N
E N
N N
The Queue is Empty!
The Queue is Empty!
Checking the Front: F
Checking the Back: N
Displaying Queue Elements: F E N
PS C:\Users\TIPQC\Documents\HOA5p1\ILOB>
```

Observation:

In this code, which highlights the topic template-based linked-list queue made me understand again how important the concept of applying what we have learned from the previous lessons which are the array, linked-list, pointers, nodes, stacks, and queues in creating what we have in this code. One wrong implementation will cause the whole block of code to error. That is why it is important in this code to overlook every part in order to not miss the quotes in output statements, and to check if the enqueue is a non-empty case. In my case, I encountered a segmentation fault in the main.cpp file, the main cause is that I printed an empty queue in queueLL.h before it was implemented.

Remarks:

If I were to choose between array implementation and linked-list implementation, I would choose the linked-list implementation for convenience and optimization of code. I observed while doing the two separate files, in array-based implementation, we manually put each of the instances in the code. Meanwhile, for me, the queueLL.h uses a method abstraction to hide the complex part of code, and not to manually code each part of the lines. That's why in the main file, we just have 40 lines, and simply connect "queueLL.h" in order for this file to call out the other file. Then, we simply just print it all in the main file. That is why for me, I would choose the linked-list for a much organized use of codes.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOC (main.cpp)

main.cpp | printerjob.h | sharedprinter.h

main.cpp > main()

```cpp
#include "SharedPrinter.h"

int main()
{
    // Create the shared printer.
    Printer printer;

    // Add the first print job.
    printer.addJob(Job(101, "Joma", 10));

    // Add the second print job.
    printer.addJob(Job(102, "Carlo", 6));

    // Add the third print job.
    printer.addJob(Job(103, "Rodney", 8));

    // Process the print jobs in FIFO order.
    printer.processJobs();

    // End the program.
    return 0;
}
```

ILOC (printerjob.h)

main.cpp | printerjob.h | sharedprinter.h

printerjob.h > Job > getPages() const

```
1   #ifndef PRINTERJOB_H
2   #define PRINTERJOB_H
3
4   #include <string>
5
6   // Create the Job class to store a print request.
7   class Job{
8   private:
9       // Store the job ID.
10      int id;
11
12      // Store the user's name.
13      std::string name;
14
15      // Store the number of pages.
16      int pages;
17
18  public:
19
20      // Initialize default values.
21      Job(){
22          id = 0;
23          name = "";
24          pages = 0;
25      }
```

main.cpp | printerjob.h | sharedprinter.h

printerjob.h > Job > getPages() const

```
1   #ifndef PRINTERJOB_H
7   class Job{
21      Job(){
26
27      // Initialize the job with user-provided values.
28      Job(int jobId, std::string user, int numPages){
29          id = jobId;
30          name = user;
31          pages = numPages;
32      }
33
34      // Return the job ID.
35      int getId() const{
36          return id;
37      }
38
39      // Return the user's name.
40      std::string getName() const{
41          return name;
42      }
43
44      // Return the number of pages.
45      int getPages() const{
46          return pages;
47      }
48
49  };
50
51  #endif
```

ILOC (sharedprinter.h)

main.cpp | printerjob.h | sharedprinter.h

sharedprinter.h > Printer > isFull()

```
1   #ifndef SHAREDPRINTER_H
2   #define SHAREDPRINTER_H
3
4   #include <iostream>
5   #include "PrinterJob.h"
6
7   // Create the Printer class.
8   class Printer{
9   private:
10
11      // Set the maximum number of print jobs.
12      static const int MAX = 5;
13
14      // Create the queue using an array.
15      Job queue[MAX];
16
17      // Track the first job in the queue.
18      int front;
19
20      // Track the last job in the queue.
21      int rear;
22
23  public:
24
25      // Initialize an empty queue.
```

main.cpp | printerjob.h | sharedprinter.h

sharedprinter.h > Printer > isFull()

```
1   #ifndef SHAREDPRINTER_H
8   class Printer{
26      Printer(){
27          front = 0;
28          rear = -1;
29      }
30
31      // Check whether the queue has no jobs.
32      bool isEmpty(){
33          return rear < front;
34      }
35
36      // Check whether the queue is full.
37      bool isFull(){
38          return rear == MAX - 1;
39      }
40
41      // Add a new print job to the rear of the queue.
42      void addJob(Job job){
43          // Check if the queue is already full.
44          if (isFull()){
45              std::cout << "Printer queue is full.\n";
46              return;
47          }
```

main.cpp | printerjob.h | sharedprinter.h

sharedprinter.h > Printer > isFull()

```
1   #ifndef SHAREDPRINTER_H
8   class Printer{
42      void addJob(Job job){
48
49          // Move the rear pointer.
50          rear++;
51
52          // Store the new job.
53          queue[rear] = job;
54
55          // Display the added user.
56          std::cout << "\n===== Added User =====\n";
57          std::cout << "Job ID : " << job.getId() << std::endl;
58          std::cout << "Name   : " << job.getName() << std::endl;
59          std::cout << "Pages  : " << job.getPages() << std::endl;
60
61          // Display the current queue.
62          std::cout << "\nCurrent Queue (First Come, First Serve)\n";
63
64          for (int i = front; i <= rear; i++){
65              std::cout << queue[i].getId()
66                        << " - "
67                        << queue[i].getName()
68                        << " ("
69                        << queue[i].getPages()
```

main.cpp | printerjob.h | sharedprinter.h

sharedprinter.h > Printer > isFull()

```
1   #ifndef SHAREDPRINTER_H
8   class Printer{
42      void addJob(Job job){
64          for (int i = front; i <= rear; i++){
70                        << " pages)"
71                        << std::endl;
72          }
73
74          std::cout << std::endl;
75      }
76
77      // Process all jobs using FIFO.
78      void processJobs(){
79          // Check if there are no jobs.
80          if (isEmpty()){
81              std::cout << "No jobs to process.\n";
82              return;
83          }
84
85          std::cout << "\n===== Processing Jobs =====\n";
86
87          // Continue until every job has been processed.
88          while (!isEmpty()){
89              // Print the first job in the queue.
90              std::cout << "\nPrinting Job "
```

main.cpp | printerjob.h | sharedprinter.h

sharedprinter.h > Printer > isFull()

```
 1  #ifndef SHAREDPRINTER_H
 8  class Printer{
78      void processJobs(){
88          while (!isEmpty()){
91                      << queue[front].getId()
92                      << " - "
93                      << queue[front].getName()
94                      << " ("
95                      << queue[front].getPages()
96                      << " pages)"
97                      << std::endl;
98
99              // Remove the processed job.
100             front++;
101
102             // Display the queue after processing.
103             std::cout << "\nQueue After Processing\n";
104
105             if (isEmpty()){
106                 std::cout << "Queue is empty.\n";
107             }else{
108                 for (int i = front; i <= rear; i++)
109                 {
110                     std::cout << queue[i].getId()
111                               << " - "
```

main.cpp | printerjob.h | sharedprinter.h

sharedprinter.h > Printer > isFull()

```
 1  #ifndef SHAREDPRINTER_H
 8  class Printer{
78      void processJobs(){
88          while (!isEmpty()){
107             }else{
112                             << queue[i].getName()
113                             << " ("
114                             << queue[i].getPages()
115                             << " pages)"
116                             << std::endl;
117                 }
118             }
119         }
120
121         // Display the final queue.
122         std::cout << "\n===== Final Queue =====\n";
123         std::cout << "Queue is empty.\n";
124
125         // Display completion message.
126         std::cout << "\nAll jobs completed.\n";
127     }
128 };
129
130 #endif
```

ILOC (CODE OUTPUT):

```
cuments\SUPP_REYES_QUEUES\main.exe"

===== Added User =====
Job ID : 101
Name   : Joma
Pages  : 10

Current Queue (First Come, First Serve)
101 - Joma (10 pages)


===== Added User =====
Job ID : 102
Name   : Carlo
Pages  : 6

Current Queue (First Come, First Serve)
101 - Joma (10 pages)
102 - Carlo (6 pages)


===== Added User =====
Job ID : 103
Name   : Rodney
Pages  : 8

Current Queue (First Come, First Serve)
101 - Joma (10 pages)
102 - Carlo (6 pages)
103 - Rodney (8 pages)
```

```
===== Processing Jobs =====

Printing Job 101 - Joma (10 pages)

Queue After Processing
102 - Carlo (6 pages)
103 - Rodney (8 pages)

Printing Job 102 - Carlo (6 pages)

Queue After Processing
103 - Rodney (8 pages)

Printing Job 103 - Rodney (8 pages)

Queue After Processing
Queue is empty.

===== Final Queue =====
Queue is empty.

All jobs completed.
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\SUPP_REYES_QUEUES>
```

CODE ANALYSIS:

This software program simulates the functioning of a shared printer where printing jobs are processed by implementing the array-based queue concept. The class used here is Job which has all the information required to process a print job such as job ID, name of the person requesting the printing, number of pages, etc. The Printer class here is used to work as a queue manager. The jobs are then added to the queue as they come. Each time a new job is added to the system, the program displays the queue state. When the processing of jobs begins, the printer deletes the job from the queue. Thus, it illustrates the principle of FIFO (First In First Out) where the job printed before will always be executed first. It also shows how array and queue concepts can be applied to real life scenarios with C++.

WHY DID YOU USE ARRAYS OR LINKED LIST?

An array was chosen because the printer queue has a fixed maximum capacity, making it a simple and efficient way to store print jobs. Furthermore, arrays are easy to implement, require less memory overhead than linked lists, and allow queue operations such as adding and processing jobs to be performed efficiently for a small number of elements such as what I have done in this work. Since this activity only requires a basic simulation of a shared printer, an array-based implementation is sufficient and makes the program easier to understand and maintain.

OUTPUT ANALYSIS:

The output follows the First In, First Out (FIFO) rule of a queue. When print jobs are sent by Joma, Carlo, and Rodney, these jobs are inserted into the rear end of the queue, and the current queue represents the sequence of the print requests pending. During this whole process, the printer always prints the job in the front of the queue first before discarding it, allowing the printer to move on to the next job. Accordingly, one sees the queue getting smaller after each print job is finished even to the point of being empty at the end. Such behavior illustrates the work of a common office printer, which processes requests in the order received, and thus, provides fair service to all users.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, Explanation of Errors, and Code Debugging**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI only assisted in recommendation of concepts to be used and assisted with debugging, and improving code organization. All final decisions, understanding, testing, and completion of this work were done by me, and AI did not replace my original thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.